ShopHub AI

Complete FYP Development Plan - Version 2 (with additions)

Personalized Shopping and Intelligent Product Discovery. A MERN e-commerce platform with a database-grounded bilingual AI shopping assistant, explainable recommendations, review analysis, and a measured research evaluation.

Legend: **[NEW]** added in v2 **[FIX]** corrected from v1. Unmarked items are carried over from the original plan.

1. Executive Summary

ShopHub AI is a full-stack e-commerce FYP built with React, Node.js/Express and MongoDB. Its academic contribution is an AI shopping experience grounded in real product data that gives explainable recommendations and measures whether explanations improve perceived relevance, trust, usefulness and click-through rate (CTR).

Priority order: core e-commerce first, then AI assistant and explainable recommendations, then research evaluation. Human support chat, Redis, visual search and forecasting are optional and must not delay the core deliverables.

2. What Is New in v2

#	Addition	Why it matters
1	**[NEW]** Missing original features added: discounts/coupons, product comparison, returns/refunds, contact form, inventory alerts	Your original idea listed these, but v1 had no collections or APIs for them.
2	**[FIX]** Experiment design made consistent (within-subject, counterbalanced) and statistics matched to it	v1 mixed between-subject assignment with paired tests, which is statistically invalid.
3	**[NEW]** Security hardening: validation, rate limiting, Helmet, NoSQL-injection sanitizing, prompt-injection defense	Examiners and viva questions often target security and AI safety.
4	**[NEW]** AI evaluation: bilingual test set, grounding accuracy, explanation-faithfulness check, offline recommender metrics	Gives measurable results beyond the user survey.
5	**[NEW]** Seed data and demo plan (200+ products, synthetic interactions, clearly disclosed)	Recommendations and review analysis need data to demonstrate.
6	**[NEW]** Payments detail: sandbox gateway, idempotency, order status lifecycle	Prevents duplicate orders and defines clear states.
7	**[NEW]** AI cost, logging and fallback controls	Protects your API budget and keeps the demo working if the LLM fails.
8	**[NEW]** Research ethics and consent checklist	Needed for a user study with real participants.
9	**[NEW]** 14-week timeline, risk register, documentation list, viva Q&A;	Turns the plan into a schedule and report outline.

3. Phase 1 - Technology Stack and Project Setup

Component	Technology
Frontend	React + Vite + Tailwind CSS
Backend	Node.js + Express.js
Database	MongoDB Atlas + Mongoose
Authentication	JWT + bcrypt
AI	OpenAI or Claude API with tool/function calling
Real-time chat	Socket.IO on a persistent host
Testing	Postman + Jest + Supertest
Frontend deployment	Vercel
Backend deployment	Render, Railway or DigitalOcean App Platform
[NEW] Validation	Zod or Joi for every request body/query
[NEW] Security middleware	helmet, express-rate-limit, express-mongo-sanitize, cors allow-list
[NEW] API documentation	Swagger / OpenAPI (swagger-ui-express)
[NEW] Payments	Stripe test mode or JazzCash/Easypaisa sandbox, plus COD
Optional	Redis, Docker, Python FastAPI (only if a separate ML/forecast service is truly needed)

Folder structure

```
shophub-ai/
  client/src/ (components, pages, layouts, services, hooks, context)
  server/src/ (config, models, controllers, routes, middleware,
               services/{ai,recommendations,reviews}, sockets, utils, app.js)
  research/ (experiment-plan.md, consent-form.md, results/)
  docs/ (SRS, ERD, use-cases, sequence diagrams)        <- NEW
  scripts/ (seed.js, eval-ai.js, eval-recs.js)            <- NEW
  .env.example  .gitignore  README.md
```

Store API keys, JWT secrets and database credentials in .env files. Never commit secrets. **[NEW]** Commit a .env.example with placeholder values, and rotate any key that was ever pushed by mistake.

4. Phase 2 - MongoDB Collections

18 collections: the original 14 plus 4 new ones for features that were in your idea but missing from v1. Add research and forecasting collections only when those features are built.

#	Collection	Important fields
1	users	name, email, passwordHash, role, preferences, createdAt
2	categories	name, slug, parentId, isActive
3	products	name, description, categoryId, price, specifications, images, stock, lowStockThreshold, isActive, embedding. **[NEW]** Add a text index on name + description.
4	carts	userId, items, updatedAt
5	wishlists	userId, productIds
6	orders	userId, items (snapshot), total, discountTotal, couponCode, paymentMethod, paymentStatus, orderStatus, address, statusHistory, idempotencyKey
7	reviews	userId, productId, rating, comment, verifiedPurchase, createdAt
8	conversations	userId, language, title, createdAt
9	messages	conversationId, role, content, productReferences, createdAt

#	Collection	Important fields
10	recommendations	userId, productIds, reasons, method, experimentGroup, createdAt
11	supportConversations	userId, agentId, status, createdAt
12	supportMessages	conversationId, senderId, message, createdAt
13	notifications	userId, type, title, message, readAt, createdAt
14	reviewAnalyses	productId, aspects, pros, cons, summary, reviewCount, analyzedAt
15	**[NEW]** coupons	code, type (percent/fixed), value, minOrder, maxUses, usedCount, expiresAt, isActive
16	**[NEW]** returnRequests	orderId, userId, items, reason, status, refundAmount, createdAt
17	**[NEW]** contactMessages	name, email, subject, message, status, createdAt
18	**[NEW]** aiLogs	userId, conversationId, model, inputTokens, outputTokens, latencyMs, toolCalls, refused, createdAt
Opt.	researchEvents	participantCode, experimentGroup, eventType, recommendationId, eventId, timestamp
Opt.	salesForecasts	forecastPeriod, predictedSales, modelMetadata, evaluationResults

Order items store product name, purchase price and quantity as a snapshot so history stays correct when prices change. Passwords are stored only as bcrypt hashes. Store money as integer paisas or Decimal128. Role-based access is enforced on the backend.

Core Design Decisions

- **Database-grounded AI:** retrieve real MongoDB records before answering; never invent prices, stock or specs; say so when data is unavailable.
- **Explainable recommendations:** every reason is verified against actual product attributes and user preferences.
- **Research instrumentation from day one:** log impressions, clicks, experiment group and server-generated timestamps.
- **[NEW] Measure the AI itself:** keep a fixed test set of queries (English, Urdu, Roman Urdu) and re-run it after every AI change.
- **[NEW] Cost and safety controls:** per-user daily AI limit, token logging, and a non-AI fallback if the LLM is down.

5. Phase 3 - Backend APIs and Business Logic

Module	Main endpoints
Auth	POST /api/auth/register, POST /api/auth/login, GET /api/auth/me
Products	GET /api/products, GET /api/products/:id, POST, PATCH /:id, DELETE /:id
Categories	GET /api/categories, POST, PATCH /:id, DELETE /:id
Cart	GET /api/cart, POST /api/cart/items, PATCH and DELETE /api/cart/items/:productId
Wishlist	GET /api/wishlist, POST and DELETE /api/wishlist/:productId
Orders	POST /api/orders, GET /api/orders/my, GET /api/orders/:id, PATCH /api/admin/orders/:id/status
Reviews	GET and POST /api/products/:id/reviews
AI assistant	POST /api/ai/chat, GET /api/ai/conversations, GET /api/ai/conversations/:id/messages
Recommendations	GET /api/recommendations, POST /api/research/events
Review analysis	GET /api/products/:id/review-analysis, POST /api/admin/products/:id/analyze-reviews
Notifications	GET /api/notifications, PATCH /api/notifications/:id/read
Admin	GET /api/admin/dashboard, /users, /sales
[NEW] Coupons	POST /api/cart/coupon (validate), admin CRUD /api/admin/coupons
[NEW] Comparison	GET /api/compare?ids=a,b,c (specs table) and POST /api/ai/compare (AI summary from retrieved data)
[NEW] Returns	POST /api/orders/:id/return, PATCH /api/admin/returns/:id
[NEW] Contact	POST /api/contact, GET /api/admin/contact
[NEW] Inventory	GET /api/admin/inventory/low-stock
[NEW] Health	GET /api/health (for deployment checks)

Every route requires validation, authentication, authorization, error handling and correct HTTP status codes. Admin-only operations are protected by backend middleware, not just hidden in the UI. **[NEW]** Use one central error handler and a consistent JSON error format.

Order Placement Rules

- Never accept prices from the client; read the current price from MongoDB at checkout.
- Validate stock before creating an order, and save a purchase-time price snapshot.
- Use atomic, concurrency-safe stock updates inside a transaction.
- COD orders: paymentMethod = COD, paymentStatus = PENDING. Only admins change order status.
- **[NEW] Idempotency:** the client sends an idempotencyKey; a repeated key returns the existing order instead of creating a duplicate.
- **[NEW] Coupons:** validate code, expiry, minimum order and usage limit on the server; recompute totals server-side.
- **[NEW] Status lifecycle:** PLACED -> CONFIRMED -> SHIPPED -> DELIVERED, with CANCELLED (before shipping) and RETURN_REQUESTED -> REFUNDED. Reject invalid transitions and restore stock on cancellation.
- **[NEW] Payments:** for online payment, confirm via the gateway webhook (verify the signature) rather than trusting the browser redirect.

```
await Product.findOneAndUpdate(
  { _id: item.productId, stock: { $gte: item.quantity } },
  { $inc: { stock: -item.quantity } },
  { session }
);
```

Security Checklist [NEW]

- Validate all input with Zod/Joi; sanitize against NoSQL injection (express-mongo-sanitize).
- helmet headers, strict CORS allow-list, rate limits on login and AI endpoints.
- Short-lived JWT access token; httpOnly cookie or carefully handled storage; bcrypt cost 10-12.
- Never log passwords or tokens; do not send personal data (address, phone) to the LLM.
- Upload limits and type checks for product images; use Cloudinary or similar.

6. Phase 4 - AI Assistant, Recommendations and Review Analysis

AI Shopping Assistant (RAG)

Supports English, Urdu and Roman Urdu. Flow: understand the query, extract constraints (budget, category, specs) with tool/function calling, run a clean MongoDB query, pass only the retrieved records to the LLM, and answer with product links. If data is missing, say so.

Example: 'Mujhe 80,000 ke andar 16GB RAM wala laptop dikhao' becomes category = laptops, maxPrice = 80000, specifications.ram = 16GB. MongoDB filters and text search are enough for v1; Atlas Vector Search is optional (keep the embedding field).

Guardrails [NEW]

- **Prompt-injection defense:** treat product descriptions and reviews as data, never as instructions; the system prompt states this, and tool arguments are re-validated by the backend.
- **Scope control:** refuse non-shopping topics politely; never reveal system prompts or other users' data.
- **Price/stock check:** after generation, verify that every price, stock claim and product ID in the answer exists in the retrieved records; if not, regenerate or fall back to a plain product list.
- **Fallback:** if the LLM fails or times out, return filtered products with a simple template message.
- **Limits:** max tokens per reply, a per-user daily message cap, and aiLogs for tokens, latency and refusals.
- **Order-status questions** use an authenticated tool that only returns the logged-in user's own orders.

AI Evaluation Plan [NEW]

Test	Method	Report
Query understanding	30-50 labelled queries across English, Urdu and Roman Urdu; compare extracted parameters to expected	Accuracy per language
Grounding	Check every price/stock/spec in responses against the database	% of responses fully grounded
Out-of-domain refusal	10-15 off-topic or injection-style prompts	Refusal rate
Latency and cost	From aiLogs	Average latency, tokens per chat

Personalized and Explainable Recommendations

Start with an explainable content-based engine: filter eligible products, rank by user preferences and interaction history, and return structured reasons with every result.

- Budget match; category and product preference match; required specifications (RAM, storage, processor, screen size).
- Stock availability; previous views, wishlist items and purchases; popularity/category fallback for new users (cold start).
- Cosine similarity over product feature vectors is the content-based baseline. Explanation types: feature-match, value/budget, social-proof.
- [NEW] **Explanation faithfulness:** generate reasons from rules/templates over real attributes (the LLM may only rephrase them), so a reason can never claim something the data does not support.
- [NEW] **Offline metrics:** compare against a popularity baseline using Precision@K, Recall@K and catalogue coverage on held-out interactions.
- [NEW] **Tracking:** log recommendation views, wishlist adds and purchases as implicit feedback.

```
{ "productId": "PRODUCT_ID",
  "reasons": ["Price is within your budget", "Matches your requested RAM", "Available in stock"],
  "method": "preference_match" }
```

Smart Product Comparison [NEW]

Users select 2-4 products. The backend builds a specification table from the database; the LLM then writes a short comparison (price, performance, battery, best for whom) using only that table. This feature was in your original idea and is now part of the plan.

Customer Review Analysis

Aspect-based sentiment analysis, cached in reviewAnalyses and refreshed when new reviews arrive or an admin triggers it. Do not call the LLM on every page view. If a product has few or no reviews, show the uncertainty clearly instead of an unreliable summary. **[NEW]** Set a minimum (for example 5 reviews) before showing a summary, and show the number of reviews analysed.

```
{ "productId": "...", "aspects": { "battery": {"sentiment": "positive", "score": 85},
  "buildQuality": {"sentiment": "mixed", "score": 50} },
  "summary": "...", "analyzedReviewCount": 24, "lastAnalyzedAt": "..." }
```

Seed Data Plan [NEW]

- At least 200 products across 6-8 categories (laptops, phones, headphones, etc.) with realistic specifications and PKR prices.
- Around 30-50 demo users, a few hundred interactions and 300+ reviews; if synthetic, say so clearly in the report.
- A scripts/seed.js that resets and rebuilds the demo data in one command.

7. Phase 5 - Real-Time Features, Notifications and Redis

Implement only after the core e-commerce and AI features are stable.

Socket.IO Support Chat

- Users open a support conversation; agents join authorized conversations only.
- Messages are delivered in real time and persisted in MongoDB for history.
- The server verifies permissions on every join and message.

Order Notifications

- Create and persist a notification on every order status change; deliver live via Socket.IO when connected.
- **[NEW]** Optional email notification (Nodemailer) for order confirmation.

Redis Caching

- Cache public product and category data; invalidate on product updates.
- Never put user-specific data into a shared cache; fall back safely if Redis is unavailable.

Sales forecasting is optional. Do not add it without a reliable historical dataset; synthetic-data forecasting is hard to defend in a viva. **[NEW]** A safer alternative is a simple admin analytics dashboard (sales by day, top products, low stock) built from real order data.

Visual Search [NEW] (stretch only)

If everything else is complete: embed product images with an image-embedding model, store vectors, and return the nearest products for an uploaded photo. Skip it if time is short.

8. Phase 6 - Research Experiment and CTR

Research question: Does providing explanations with recommendations improve perceived relevance, trust and usefulness compared with recommendations without explanations?

Experiment Design

Item	Plan
Participants	About 15-20 (aim for 20+ if possible), subject to supervisor approval
[FIX] Design	**Within-subject, counterbalanced.** Every participant sees both conditions on different but comparable shopping tasks; half see A first, half see B first. v1 mixed between-subject assignment with paired tests, which does not fit.
Condition A	Recommendations without explanations
Condition B	Comparable recommendations with explanations
Ratings	Relevance, trust, usefulness on a 1-5 Likert scale (2-3 items per construct, averaged)
Behavioural metric	CTR = (recommendation clicks / impressions) x 100
[NEW] Extra measures	Task completion time, number of products viewed before choosing, and one open-ended comment question
Privacy	Pseudonymous participant codes; avoid unnecessary personal data

```
{ "eventId": "uuid", "participantCode": "P001", "experimentGroup": "B",
  "eventType": "recommendation_click", "recommendationId": "ID",
  "timestamp": "SERVER_GENERATED" }
```

Define what counts as an impression (for example, a card visible for at least 1 second) and a click, and apply the rule consistently. Use server timestamps and unique event IDs, and ignore duplicate events.

Statistical Analysis

- Report means, medians and standard deviations for relevance, trust and usefulness in both conditions.
- **[FIX] Ratings:** Wilcoxon signed-rank test (preferred for small Likert samples) or paired t-test if differences look normal. Report the effect size and a confidence interval.
- **[FIX] CTR:** compute CTR per participant per condition and compare with Wilcoxon. A chi-square (or Fisher's exact) test on pooled clicks is acceptable only as a secondary check, because clicks from the same person are not independent.
- One chart per metric (relevance, trust, usefulness, CTR); state sample size and limitations and avoid universal claims.
- Tools: Python (scipy, pandas, matplotlib) or Excel for the analysis and charts.

Research Ethics Checklist [NEW]

- Get supervisor / department approval before recruiting participants.
- Short consent form: purpose, voluntary participation, what is recorded, right to withdraw, no personal data stored.
- Store only participant codes; keep the code-to-person list (if any) separate and delete it afterwards.
- Do not use real payment details in the experiment; use demo accounts only.

9. Phase 7 - Testing, Deployment and Final Report

Testing Checklist

- Registration, login, invalid credentials and session handling; user/admin permissions on protected endpoints.
- Product CRUD, search, filters and pagination; cart totals, price validation, stock validation.
- Duplicate-order prevention (idempotency), COD status transitions, coupon rules, return flow.
- Review validation and verified-purchase rules.
- AI grounding, bilingual input, out-of-domain refusal, **[NEW]** prompt-injection attempts and LLM-failure fallback.
- Recommendation explanations match product data; Socket.IO access control and message persistence.
- Accurate recording of research impressions and clicks.
- **[NEW]** **Security tests:** NoSQL-injection payloads, brute-force rate limiting, accessing another user's order, calling admin routes as a normal user.
- **[NEW]** **Concurrency test:** two simultaneous orders for the last item in stock; only one may succeed.
- **[NEW]** **Frontend:** mobile responsiveness and basic accessibility (labels, contrast, keyboard use).

Deployment Steps

- Production MongoDB Atlas cluster (restrict IP access, separate DB user).
- Deploy the backend with environment variables; deploy the frontend on Vercel.
- Configure CORS, HTTPS and production API URLs; secure all keys.
- **[NEW]** Set up GitHub Actions to run Jest tests on every push.
- **[NEW]** Use the /api/health endpoint and an uptime monitor; note that free hosts may sleep, so wake the server before the demo.
- Run Postman smoke tests after deployment; prepare README, screenshots and demo data.
- **[NEW]** Record a 3-5 minute backup demo video in case of internet or API problems on viva day.

10. Priority Roadmap

Priority	Phase	Core deliverables
P0 - Must	Core e-commerce	JWT auth + RBAC, product CRUD/search/filters, cart, concurrency-safe COD checkout, orders, admin panel, coupons
P0 - Must	AI assistant (RAG)	Tool-calling parser, MongoDB retrieval, grounded response, guardrails, saved conversations
P0 - Must	Explainable recommendations	Content-based filtering, structured reasons, research event tracking
P1 - High	Research and analysis	15-20 participant experiment, Likert survey, CTR, statistical tests, charts
P1 - High	**[NEW]** Comparison, review analysis, returns	Smart comparison, cached review summaries, return/refund flow
P2 - Nice	Real-time and optimization	Order notifications, Redis, live support chat
P3 - Stretch	Visual search, forecasting	Only if everything above is complete and stable

11. Suggested 14-Week Timeline [NEW]

Weeks	Focus	Output
1	Requirements, ERD, setup, repo	SRS draft, ERD, project skeleton
2-3	Auth, RBAC, products, categories, search	Working catalogue and admin product CRUD
4-5	Cart, checkout, orders, coupons, payments sandbox	Complete purchase flow with tests

Weeks	Focus	Output
6	Admin dashboard, inventory, notifications, contact, returns	Full core store (code freeze for features)
7-8	AI assistant (RAG), guardrails, bilingual test set	Grounded chatbot with evaluation results
9-10	Recommendations, explanations, comparison, review analysis	Explainable recommender + offline metrics
11	Research instrumentation, pilot with 2-3 users	Event logging verified
12	Main user study (15-20 users) and analysis	Charts and statistical results
13	Testing, security checks, deployment, bug fixes	Live deployed system
14	Report, slides, demo video, viva practice	Final submission

12. Risk Register [NEW]

Risk	Mitigation
Scope too large	Freeze features at week 6; treat P2/P3 as optional
LLM API cost or outage	Daily caps, caching, non-AI fallback, backup demo video
Too few study participants	Recruit early (classmates, other departments); use within-subject design so fewer are needed
Hallucinated product data	RAG only, post-generation price/stock verification, grounding tests
Free hosting sleeps or slows	Wake server before demo; keep a local copy ready
Payment gateway approval delays	Use sandbox/test mode and COD; state this limitation in the report
Team or time loss	Weekly milestones in GitHub Projects; commit daily

13. Report and Documentation Checklist [NEW]

- SRS: functional and non-functional requirements; use-case diagram (Admin, Customer, Support Agent).
- ERD for the MongoDB collections, architecture diagram, and sequence diagrams for checkout and the AI chat flow.
- API documentation (Swagger), test evidence (Jest and Postman screenshots), deployment guide.
- Research chapter: question, method, ethics, results, charts, limitations, future work.
- Literature review: recommender systems, explainable AI, RAG, e-commerce chatbots.

14. Likely Viva Questions [NEW]

- How do you stop the chatbot from inventing prices or products? (RAG, tool calling, post-check, tests.)
- How do you know the explanations are true? (Reasons come from verified attributes, not free LLM text.)
- Why this statistical test, and is 15-20 users enough? (Within-subject, non-parametric test, limitations stated.)
- How do you prevent overselling or duplicate orders? (Atomic stock update, transactions, idempotency key.)
- How is user data protected? (bcrypt, JWT, RBAC, validation, no personal data sent to the LLM, pseudonymous study.)
- What happens if the LLM or Redis is down? (Fallbacks.)
- What is new compared with existing platforms such as Daraz? (Grounded bilingual assistant plus measured explainability.)

15. Final Deliverables

- Working e-commerce application and admin dashboard.

- Database-grounded bilingual AI shopping assistant with evaluation results.
- Personalized and explainable recommendation system, smart comparison, and review summaries.
- Research experiment with comparison charts and statistical analysis.
- Postman and Jest test evidence, security test notes.
- Final report, presentation, deployment documentation and backup demo video.